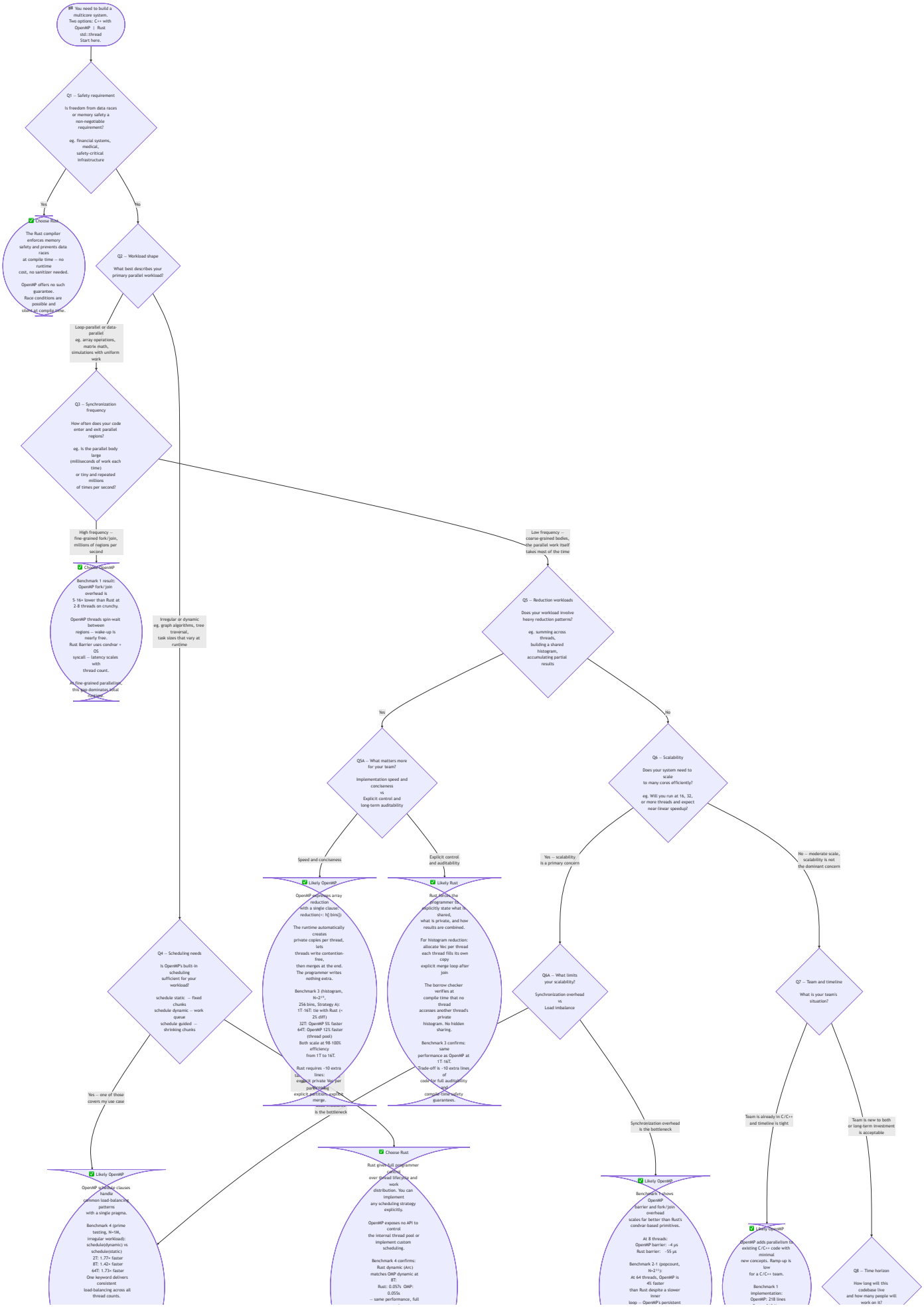
Decision Flowchart: C++/OpenMP vs Rust for Multicore Systems

> **How to use this flowchart:** Walk through each question from top to bottom. Answer based on your system's requirements. Each terminal outcome cites which benchmark provides the supporting evidence. Nodes marked **[TBD]** will be updated as Benchmarks 2–4 are completed.

Flowchart


You need to build a multicore system. Two options: C++ with OpenMP | Rust std::thread Start here.
Q1 — Safety requirement
Is freedom from data races or memory safety a non-negotiable requirement?
eg. financial systems, medical, safety-critical infrastructure
Yes
No
Choose Rust
The Rust compiler enforces memory safety and prevents data races at compile time — no runtime cost, no sanitizer needed.
OpenMP offers no such guarantee. Race conditions are possible and silent at compile time.
Q2 — Workload shape
What best describes your primary parallel workload?
Loop-parallel or data-parallel eg. array operations, matrix math, simulations with uniform work
Irregular or dynamic eg. graph algorithms, tree traversal, task sizes that vary at runtime
Q3 — Synchronization frequency
How often does your code enter and exit parallel regions?
eg. Is the parallel body large (milliseconds of work each time) or tiny and repeated millions of times per second?
High frequency — fine-grained fork/join, millions of regions per second
Choose OpenMP
Benchmark 1 result: OpenMP fork/join overhead is 5-16× lower than Rust at 2-8 threads on crunchy.
OpenMP threads spin-wait between regions — wake-up is nearly free. Rust Barrier uses condvar + OS syscall — latency scales with thread count.
At fine-grained parallelism, this gap dominates total runtime.
Low frequency — coarse-grained bodies, the parallel work itself takes most of the time
Q5 — Reduction workloads
Does your workload involve heavy reduction patterns?
eg. summing across threads, building a shared histogram, accumulating partial results
Yes
No
Q5A — What matters more for your team?
Implementation speed and conciseness vs Explicit control and long-term auditability
Speed and conciseness
Explicit control and auditability
Likely OpenMP
OpenMP expresses array reduction with a single clause: reduction(+: h[:bins])
The runtime automatically creates private copies per thread, lets threads write contention-free, then merges at the end. The programmer writes nothing extra.
Benchmark 3 (histogram, N=2^16, 256 bins, Strategy A): 1T: 16T: tie with Rust (+ 2% diff)
32T: OpenMP 5% faster
64T: OpenMP 12% faster (thread pool)
Both scale at 98-100% efficiency from 1T to 16T.
Rust requires ~10 extra lines: explicit private Vec per parallelism, explicit partition, explicit merge.
Likely Rust
Rust forces the programmer to explicitly state what is shared, what is private, and how results are combined.
For histogram reduction: allocate Vec per thread each thread fills its own copy explicit merge loop after join
The borrow checker verifies at compile time that no thread accesses another thread's private histogram. No hidden sharing.
Benchmark 3 confirms: same performance as OpenMP at 1T-16T.
Trade-off is ~10 extra lines of code for full auditability and compile-time safety guarantees.
Q4 — Scheduling needs
Is OpenMP's built-in scheduling sufficient for your workload?
schedule static — fixed chunks
schedule dynamic — work queue
schedule guided — shrinking chunks
Yes — one of those covers my use case
Likely OpenMP
OpenMP schedule clauses handle common load-balancing patterns with a single pragma.
Benchmark 4 (prime testing, N=1M, irregular workload): schedule(dynamic) vs schedule(static)
2T: 1.77× faster
8T: 1.42× faster
64T: 1.73× faster
One keyword delivers consistent load-balancing across all thread counts.
Is the bottleneck
Choose Rust
Rust gives full programmer control over thread lifecycle and work distribution. You can implement any scheduling strategy explicitly.
OpenMP exposes no API to control the internal thread pool or implement custom scheduling.
Benchmark 4 confirms: Rust dynamic (Arc) matches OMP dynamic at 8T:
Rust: 0.057s OMP: 0.055s
— same performance, full
Q5B — Scalability
Does your system need to scale to many cores efficiently?
eg. Will you run at 16, 32, or more threads and expect near-linear speedup?
Yes — scalability is a primary concern
No — moderate scale, scalability is not the dominant concern
Q6A — What limits your scalability?
Synchronization overhead vs Load imbalance
Synchronization overhead is the bottleneck
Likely OpenMP
Benchmark 1 shows OpenMP barrier and fork/join overhead scales far better than Rust's condvar-based primitives.
At 8 threads:
OpenMP barrier: ~4 µs
Rust barrier: ~55 µs
Benchmark 2-1 (popcount, N=2^11):
At 64 threads, OpenMP is 4% faster than Rust despite a slower inner loop — OpenMP's persistent
Q7 — Team and timeline
What is your team's situation?
Team is already in C/C++ and timeline is tight
Team is new to both or long-term investment is acceptable
Likely OpenMP
OpenMP adds parallelism to existing C/C++ code with minimal new concepts. Ramp-up is low for a C/C++ team.
Benchmark 1 implementation: OpenMP: 218 lines
Q8 — Time horizon
How long will this codebase live and how many people will work on it?

Evidence Map

Each decision node in the flowchart is informed by one or more benchmarks. This table will be filled in as benchmarks are completed.

Question	Key metric	Benchmark	Status
Q3 — Sync frequency	Fork/join overhead vs thread count	Benchmark 1	✅ Complete
Q5 — Reduction workloads	Reduction LOC, runtime, correctness effort	Benchmark 3	✅ Complete
Q6A — Sync overhead at scale	Barrier cost, speedup curves	Benchmark 1 + 2-1	✅ Complete
Q6A — Load imbalance	Runtime under uneven work, scheduling	Benchmark 4	✅ Complete
Q4 — Custom scheduling	LOC, flexibility, runtime comparison	Benchmark 4	✅ Complete
Q6 — Scalability	Speedup and efficiency vs thread count	Benchmark 2-1	✅ Complete

Key Findings (Benchmark 1 — Thread Overhead)

These facts are established from the thread overhead microbenchmark (fork/join, barrier, atomic, 2–8 threads):

1. **OpenMP fork/join overhead is 5–16× lower than Rust** at 2–8 threads. → Drives Q3: if sync frequency is high, OpenMP wins.
2. **Barrier overhead is equal at 1 thread, then diverges sharply.** OpenMP: ~2–4 µs. Rust: ~16–55 µs at 2–8 threads. → Reinforces Q6A: if synchronization is the bottleneck, OpenMP scales better.
3. **Atomic increment cost is identical** (~24–102 ns on both sides, within noise). → Neither language has an advantage on hardware atomics.

4. **OpenMP provides no programmer control over thread lifecycle.** You cannot force thread creation or destruction between regions. → Drives Q4: if custom scheduling or lifecycle control is needed, OpenMP cannot do it.

5. **Rust required 95 more lines of code** and a full thread pool design for an equivalent benchmark. → Drives Q7/Q8: for rapid development, OpenMP wins on productivity.

6. **Rust's borrow checker caught all sharing bugs at compile time.** No runtime debugging needed. → Drives Q1/Q8: for safety-critical or long-lived systems, Rust's upfront cost is justified.

Key Findings (Benchmark 2 — Monte Carlo Pi)

These facts are established from the Monte Carlo Pi benchmark (Xorshift64 RNG, 1B samples, 1–64 threads):

7. **RNG choice introduced a 3× artificial performance gap.** The original comparison used mt19937_64 (C++ stdlib) vs Xorshift64 (Rust). Unifying both to Xorshift64 collapsed the gap to ~1.35×. Confounding factors in the inner loop must be eliminated before comparing parallel models. → Lesson: benchmark design determines what you are actually measuring. A 3× gap from RNG choice was masking the true parallelism comparison.

8. **With unified RNG, OpenMP is ~1.35× faster at all thread counts — a constant multiplier.** A constant ratio across all thread counts means the difference is entirely in single-thread code generation (Xorshift64's sequential dependency chain preventing SIMD), not in parallelism quality. Both models scale at the same rate. → Precursor to Benchmark 2-1: the FP and RNG inner loop must be replaced with a bias-free workload to isolate the parallelism model.

Key Findings (Benchmark 2-1 — Embarrassingly Parallel Scalability, Popcount)

These facts are established from the popcount benchmark (N = 2^{33}, 1–64 threads, both implementations clean):

9. **Both OpenMP and Rust scale at near-identical efficiency for compute-bound embarrassingly parallel work.** Both reach 97–100% parallel efficiency from 1T to 16T. Neither parallelism model has a structural scalability advantage when synchronization is absent. → Q6 confirmed: if scalability is the concern, the decision must be made on other axes.

10. **Rust/LLVM is ~1.13–1.15× faster than C++/GCC from 1T to 16T.** The gap is a **constant multiplier** — it does not change as thread count scales. This proves the difference is in single-thread code generation (LLVM's 8× loop unrolling vs GCC's scalar loop), not in the parallelism model. → Reinforces Q7/Q8: LLVM produces better code for this workload, but this does not affect the parallel scaling decision.

11. **At 64 threads, OpenMP reverses the advantage and runs 4% faster than Rust.** With 64 threads each doing only ~~250ms of compute, Rust's cost of spawning 64 fresh OS threads per trial (~~3ms total) erases its inner-loop speed advantage. OpenMP's persistent thread pool wakes up at nearly zero cost. → Reinforces Q3: fine-grained or high-thread-count parallelism favors OpenMP's thread pool model.

12. **Neither compiler uses AVX2 SIMD for popcount on crunchy.** AVX-512 VPOPCNTDQ is not available on the cluster. Both binaries use scalar `popcnt`. LLVM's advantage comes from 8× loop unrolling and multiple accumulators, not SIMD width.

Key Findings (Benchmark 2-2 — Rayon vs std::thread)

These facts are established from repeating the popcount benchmark with Rayon (N = 2^{33}, 1–64 threads):

13. **Switching from bare `std::thread` to Rayon does not fix the 64T reversal — it makes it larger.** OpenMP leads Rayon by 7.5% at 64T vs only 4% over std::thread. Rayon's work-stealing bookkeeping across 64 workers compounds the thread management overhead. → The 64T reversal is not caused by thread spawn cost alone; it is a structural property of OpenMP's persistent pool vs any fresh-thread or work-stealing model.

14. **For uniform workloads, bare `std::thread` with static partitioning beats Rayon.** std::thread is 3–5% faster than Rayon at every thread count from 1T to 32T. LLVM generates an 8× unrolled loop inside the bare thread closure vs only 4× inside Rayon's work-stealing closure. Work-stealing adds scheduler overhead that buys nothing when every element costs the same. → Rayon's advantage is for **irregular** workloads where load imbalance exists. For perfectly uniform work, static partitioning is always better.

15. **The performance ranking for uniform workloads is: std::thread > Rayon > OpenMP (1T–32T); OpenMP > std::thread > Rayon (64T).** All three scale at equivalent efficiency. The differences are in single-thread code quality and thread management overhead, not in parallelism model quality.

Key Findings (Benchmark 3 — Parallel Histogram, Reduction Workloads)

These facts are established from the histogram benchmark (N = 2^{26}, 256 bins, Strategy A, 1–64 threads):

16. **Algorithm design matters more than language choice.** Strategy B (shared atomics) made performance 4× *worse* at 64 threads. Strategy A (private histograms + merge) gives 35× speedup at 64T. Choosing the right reduction pattern is the first decision — language comes second.

17. **Performance is a tie at 1T–16T.** OpenMP and Rust differ by less than 2% at every thread count from 1T to 16T. Both scale at 98–100% parallel efficiency. The Q5A decision is entirely about code, not runtime. → Q5A confirmed: the choice between OpenMP and Rust for reduction workloads is a programmability decision, not a performance decision.

18. **OpenMP's `reduction` clause expresses the entire pattern in one line.** The programmer writes a serial-looking loop; the runtime handles private allocation, contention-free writes, and merge invisibly. Rust requires ~10 explicit lines for the same behavior. → Drives `OPENMP_REDUCE`: for teams that prioritize conciseness and development speed, OpenMP wins on this axis.

19. **Rust's explicit model gives compile-time auditability.** Every decision is visible: what each thread owns, what it writes, and when the merge happens. The borrow checker verifies at compile time that no thread accesses another's private data. → Drives `RUST_REDUCE`: for long-lived systems where correctness and code review clarity matter, Rust's verbosity is a feature.

20. **The 32T→64T cliff is a memory bandwidth wall, not a parallelism problem.** Both languages collapse from ~94% efficiency at 32T to ~50% at 64T. The 256MB input array saturates memory bandwidth at ~32 threads. This is a hardware constraint that neither language can overcome.

Key Findings (Benchmark 4 — Irregular Workload, Prime Testing)

These facts are established from the prime testing benchmark (N = 1,000,000, 5 strategies, 1–64 threads):

21. **Static scheduling causes severe, worsening load imbalance for irregular workloads.** At 8T, OpenMP static achieves only 74% parallel efficiency and Rust static achieves only 48%, because the last thread owns [875K, 1M] — the densest region of large, expensive primes. All other threads idle while it finishes. → Confirms Q6A (load imbalance): static scheduling is the wrong choice when element cost varies.

22. **Dynamic scheduling delivers consistent 1.4–1.8× speedup over static at every thread count.** OpenMP `schedule(dynamic, 100)` vs `schedule(static)`: 1.77× at 2T, 1.49× at 4T, 1.42× at 8T, 1.73× at 64T. The benefit is constant — every thread count pays the load-imbalance penalty with static, and every thread count recovers with dynamic. → Confirms Q4 (OPENMP_SCHED): for irregular workloads, OpenMP's dynamic clause is effective with minimal code change.

23. **Rust dynamic (Arc + fetch_add) matches OMP dynamic where clean.** At 8T: Rust dynamic = 0.057s, OMP dynamic = 0.055s — a **1.04× ratio**. The same load-balancing effect is achievable in Rust with an explicit shared counter. The performance is identical; the code is not. OMP: one word (`dynamic`) in one pragma. Rust: ~15 lines (Arc allocation, clone per thread, fetch_add loop). → Confirms Q4 (OPENMP_SCHED): OpenMP is more productive for this pattern.

24. **Rayon's work-stealing is finally justified for irregular workloads.** Benchmark 2-2 showed Rayon adds overhead with no benefit for uniform workloads. Here, at 16T, Rayon (0.032s) nearly matches OMP dynamic (0.028s) — a **1.14× ratio** — with zero scheduling code from the programmer. Work-stealing automatically redistributes expensive chunks that static partitioning would have bottlenecked. → Confirms Q4 (RUST_SCHED): for irregular workloads where Rayon covers the pattern, it is the highest-productivity Rust option.

25. **Scheduling choice dominates language choice for irregular workloads.** At 8T (cleanest cross-strategy comparison point): OMP dynamic: 0.055s (best) Rust dynamic: 0.057s (1.04× behind) OMP static: 0.078s (1.42× behind best) Rust static: 0.123s (2.24× behind best) The gap from wrong scheduling (2.24×) far exceeds the gap from language choice (1.04×). → Reinforces Q4 over Q7/Q8: for irregular workloads, choose the scheduling strategy first. Language is secondary.

26. **OpenMP's persistent thread pool is more robust to cluster interference than Rust std::thread.** OpenMP: 0 contaminated cells across 35 trials. Rust: multiple thread counts fully contaminated (4T static, 16T/32T dynamic, 32T rayon). Rust spawns fresh OS threads per trial — when the cluster is loaded, OS scheduling delays hit fresh threads disproportionately. This pattern has now appeared in Benchmarks 2-1, 2-2, and 4. → Reinforces Q3/Q6A: OpenMP's persistent pool is a reliability advantage on shared hardware, not just a latency advantage.

How to Read the Final Outcome

The flowchart will rarely give a perfect one-sided answer. In practice:

- If multiple paths point to **OpenMP** → OpenMP is the right fit for your system.
- If multiple paths point to **Rust** → Rust is the right fit.
- If paths are mixed → read the cited benchmark numbers for your specific bottleneck and weight the decision toward whichever axis matters most for your system.

The flowchart is a structured way to surface the right trade-off questions, not a mechanical oracle.